

National University of Computer and Emerging Sciences



Laboratory Manuals
for
Data Structures Lab
(CL-2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BDS-3B
Semester	Fall 2026

Department of Computer Science
FAST-NU, Lahore, Pakistan

Objectives

By the end of this lab, you should be able to:

- Revise core pointer concepts and pointer arithmetic.
- Allocate, use, and correctly destroy a jagged (ragged) 2-D array on the heap.
- Write generic code in C++ using function templates and class templates.
- Derive the exact operation count $f(n)$ for a piece of code and reduce it to Big-O notation.
- Verify a derived $f(n)$ experimentally by counting operations at run time.
- Implement a resizable dynamic array (the ArrayList idea) with capacity doubling, and reason about its cost.

Note on scope: Templates are introduced here as a C++ programming tool so that the data structures you build this semester work for any element type. They will be used throughout the lab, and the theory covered in class remains the basis for quizzes and exams.

1. Pointers in C++ (Recap)

You have already met pointers, so this is a fast recap before we push further.

A pointer is a variable that stores a memory address instead of a value. The `*` operator gets you the value at that address (dereferencing), and `&` gets you the address of a variable.

```
int x = 10;    // a normal variable
int* p = &x;  // p now holds the address of x
cout << *p;   // dereference p -> prints 10
```

1.1 Pointer Types

- Null pointer: deliberately points to nothing (`nullptr`); the safe way to say "not assigned yet."
- Void pointer: a generic pointer that can hold any type's address, but must be cast before use.
- Dangling pointer: still points to memory that has already been freed. Using it is undefined behaviour.
- Wild pointer: never initialized at all, points to garbage, and using it is undefined behaviour.

Rule of thumb: after every delete, set the pointer to `nullptr`. That single line converts a dangling pointer into a null pointer, which fails loudly instead of silently corrupting memory.

1.2 Pointer Arithmetic

Since a pointer just holds an address, adding or subtracting a number moves it by that many elements of its type — not by that many raw bytes.

Operation	Meaning
<code>p++ / p--</code>	move to the next / previous element
<code>p + n / p - n</code>	move n elements forward / backward
<code>*(p + n)</code>	value at the element n positions ahead (same as <code>p[n]</code>)
<code>p2 - p1</code>	number of elements between two pointers into the same block

1.3 Common Use Cases

- Dynamic memory management (new / delete).
- Passing large objects to functions without copying them.
- Building linked structures: linked lists, trees, graphs.
- Rearranging data cheaply by moving pointers instead of moving elements.

2. Dynamic Memory and Jagged Arrays

A jagged array is an array of pointers, where each pointer owns a row of a possibly different length. It is stored as `T**` — a pointer to a block of `T*` pointers.

2.1 Allocating a Jagged Array

```
int levels = 3;
int slots[] = {4, 3, 5}; // different length per row
int** grid = new int*[levels]; // 1. the array of row pointers
for (int i = 0; i < levels; i++)
    grid[i] = new int[slots[i]]; // 2. each row, individually sized
```

2.2 Releasing It Again — In Reverse Order

Every `new[]` must be matched by exactly one `delete[]`. For a jagged array you must delete the rows first; once you delete the outer array, the row pointers are gone and those rows are leaked forever.

```
for (int i = 0; i < levels; i++)
    delete[] grid[i]; // 1. each row
delete[] grid; // 2. then the array of pointers
grid = nullptr; // 3. avoid a dangling pointer
```

This clean-up belongs in the class destructor. A class that allocates memory in its constructor and does not free it in its destructor leaks memory every time an object goes out of scope. Marks are allocated for a correct destructor in this lab.

2.3 A Warning About Copying (Rule of Three)

If your class holds a raw pointer and you let C++ generate the copy constructor for you, copying an object copies the pointer, not the data. Two objects then own the same memory and both will delete it — a double free. If you need copying, you must write the copy constructor and the assignment operator yourself. For this lab, it is acceptable to disable copying instead:

```
ParkingGrid(const ParkingGrid&) = delete;
ParkingGrid& operator=(const ParkingGrid&) = delete;
```

3. Templates in C++

A template lets you write one function or class that works for any data type. The compiler generates the type-specific version for you when you actually use it.

3.1 Function Templates

```
template <class T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
```

```
}
```

T is just a placeholder — the compiler replaces it with whatever type you call the function with.

```
cout << getMax(10, 20); // T becomes int
cout << getMax(3.5, 2.1); // T becomes double
```

3.2 Multiple Template Parameters

```
template <class T1, class T2>
void showPair(T1 a, T2 b) {
    cout << a << " - " << b;
}
```

3.3 Specialization

Sometimes the generic version is not quite right for one specific type, so you write a special-case version for it. The example below needs `#include <cctype>` for `toupper`.

```
#include <cctype>

template <>
char getMax<char>(char a, char b) {
    return (toupper(a) > toupper(b)) ? a : b;
}
```

3.4 Class Templates

Whole classes can be templated too — one class definition that generates a stack, queue, or list for any type, instead of rewriting it per type.

```
template <class T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T get() { return value; }
};

Box<int> b1(10);
Box<string> b2("Hello");
```

Multiple parameters work the same way:

```
template <class T1, class T2>
class Pair {
    T1 first;
    T2 second;
public:
    Pair(T1 a, T2 b) : first(a), second(b) {}
};

Pair<int, string> p(1, "one");
```

Important — a linker error you will otherwise spend an hour on. The definition of a template member function must be visible to the compiler at the point where the template is used. If you put your class in a .h file and its

member function bodies in a separate .cpp file, the code will compile but fail at link time with "undefined reference". Keep template class definitions and their member function bodies in the same file.

3.5 Combining Pointers and Templates

A template function can take pointers as parameters, so one swap() works for int*, double*, or any type you throw at it.

```
template <class T>
void swapPtr(T* a, T* b) {
    T temp = *a;
    *a = *b;
    *b = temp;
}
```

Why bother? One class body serves many data types; the compiler checks types for you (unlike void*); and there is no cast overhead, because a clean version is generated per type.

4. Time Complexity: from f(n) to Big-O

This section connects directly to the analysis you did in the lecture. f(n) is the exact number of basic operations a piece of code performs for an input of size n. Big-O is what is left after you discard the constants and keep only the fastest-growing term.

4.1 Worked Example

```
int sum = 0;           // 1 operation
for (int i = 0; i < n; i++) // n iterations
    sum += a[i];       // 1 operation per iteration
cout << sum;           // 1 operation
```

$f(n) = 1 + n + 1 = n + 2 \rightarrow$ drop the constant $\rightarrow O(n)$

4.2 Reduction Rules

- Constant factors are dropped: $5n$ becomes $O(n)$.
- Lower-order terms are dropped: $n^2 + 3n + 7$ becomes $O(n^2)$.
- Sequential blocks add: $O(n)$ followed by $O(m)$ is $O(n + m)$.
- Nested loops multiply: an $O(n)$ loop inside an $O(m)$ loop is $O(n \cdot m)$.
- Halving or doubling the loop variable gives a logarithmic term: $O(\log n)$.

4.3 Growth Rates You Should Recognise

Big-O	Name	Typical source
$O(1)$	Constant	Swapping two pointers; array indexing
$O(\log n)$	Logarithmic	Loop variable doubles or halves
$O(n)$	Linear	Single pass over n elements
$O(n \log n)$	Linearithmic	Efficient sorting
$O(n^2)$	Quadratic	Nested pass over the same n elements

Lab Tasks

Tasks 0, 1 and 2 are to be completed during the lab session. Task 3 may be completed at home and submitted by the deadline announced.

Task 0 — Complexity Warm-Up (Pen and Paper) [10 marks]

For each snippet, write the exact operation count $f(n)$ and then reduce it to Big-O. Count one "operation" per assignment, comparison, or arithmetic statement inside the loop body. Show your working; an answer with no derivation earns no marks. Hand this sheet to your before you start coding.

(a)

```
for (int i = 0; i < n; i++)  
    cout << a[i];
```

(b)

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        cout << a[i][j];
```

(c)

```
for (int i = 1; i <= n; i = i * 2)  
    cout << i;
```

(d)

```
for (int i = 0; i < n; i++) cout << a[i];  
for (int j = 0; j < m; j++) cout << b[j];
```

(e)

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < i; j++)  
        cout << i << j;
```

For (e), do not simply write $O(n^2)$. First derive the exact count as a summation, simplify it to a closed form, and only then state the Big-O.

Task 1 — Row Swap by Pointer, Not by Value [15 marks]

(a) Write a templated function that swaps two entire rows of a jagged array by swapping the row pointers themselves, not by copying elements one at a time. Swap the matching rowSizes entries as well.

```
template <class T>  
void swapRows(T** grid, int* rowSizes, int r1, int r2);
```

(b) Now write a second function that swaps the same two rows by copying element by element:

```
template <class T>  
void swapRowsByCopy(T** grid, int* rowSizes, int r1, int r2);
```

Note that this second version only makes sense when both rows have the same length. State in a comment what your function does when they do not.

(c) Analysis. In a comment block at the top of your file, answer the following. For a row of k elements:

- Derive $f(k)$ for `swapRows` and for `swapRowsByCopy`.
- State the Big-O of each.

- Explain in one or two sentences why the pointer version is independent of the row length, while the copy version is not.
- If a row held 1,000,000 elements, roughly how many times more work would the copy version do?

Task 2 — Smart Parking Management System [45 marks]

Context

FAST-NUCES has opened a new multi-level Smart Parking Facility for students and faculty. Each level has a different number of marked slots depending on the available space, so the layout is jagged, not a neat rectangle.

The administration wants a system that records how many hours each slot was occupied today, built with dynamic, pointer-based storage and templated code instead of fixed-size arrays.

Requirements

You are the student developer intern. The facility manager gives you these requirements:

- The facility has N levels (user-defined).
- Each level may have a different number of slots (user input) — use a jagged array (T**).
- Input hourly occupancy for every slot on every level, e.g. slot 1 on level 1 was occupied for 5 hrs.
- Display the parking grid level by level, in a table.
- Calculate total occupied hours across the whole facility.
- Calculate average occupied hours per level.
- Exit option, which must free all allocated memory before the program ends.

Implementation

Use a templated class. A suggested skeleton:

```
template <class T>
class ParkingGrid {
private:
    T** grid;
    int* slotCount;
    int levels;
    bool initialized;
public:
    ParkingGrid();
    ~ParkingGrid();    // must free every row, then the outer array
    void configure();   // ask for levels and slots per level
    void inputOccupancy();
    void display() const;
    T totalHours() const;
    void averagePerLevel() const;
};
```

Initialization should look like this:

```
Enter number of levels: 3
Enter number of slots for each level:
Level 1 slots: 4
Level 2 slots: 3
Level 3 slots: 5
```

Build a menu-driven interface around the class:

```
=== Smart Parking Management ===
1. Input Slot Occupancy
2. Display Parking Grid
3. Calculate Total Occupied Hours
4. Average Occupied Hours Per Level
5. Show Operation Count (see part e)
0. Exit
Enter choice:
```

Expected Output Format for Option 2

Match this layout so that your submission can be graded consistently:

```
=== Parking Grid (hours occupied) ===
Level 1 (4 slots): 5 3 0 8
Level 2 (3 slots): 2 7 4
Level 3 (5 slots): 6 6 1 0 9
Total Occupied Hours : 51
Level 1 average : 4.00
Level 2 average : 4.33
Level 3 average : 4.40
```

(d) Validation

Your program must reject invalid input rather than crash on it. At minimum, handle: a level count of zero or less, a slot count of zero or less, negative occupancy hours, occupancy hours greater than 24, and any menu option other than 0–5. Re-prompt the user instead of terminating.

(e) Complexity Analysis and Verification

Let L be the number of levels and let n be the total number of slots in the facility (that is, the sum of the slot counts of all levels).

- Derive $f(n)$ for `totalHours()`. State its Big-O in terms of n .
- Derive the complexity of `averagePerLevel()`. Is it $O(L \times \text{maxSlots})$ or $O(n)$? Justify your answer — these are not the same thing for a jagged array, and only one of them is tight.
- Add an integer counter to your class that is incremented on every array element access inside `totalHours()`. Expose it through menu option 5.
- Run your program with three configurations: 3 levels $\times$ 4 slots each, 5 levels $\times$ 10 slots each, and 10 levels $\times$ 10 slots each. Record the counter value each time in a table in your comment block.
- Does the measured count match the $f(n)$ you derived? If it does not match exactly, explain the difference.

Task 3 — Building an ArrayList: Templated Dynamic Array [30 marks]

The parking grid you built in Task 2 is fixed once it is configured: if the administration paints one extra slot on Level 2 next week, you cannot add it without rebuilding the whole structure. A dynamic array — the idea behind ArrayList in Java and vector in C++ — solves exactly this problem, and this task asks you to build one yourself.

(a) The Class

```
template <class T>
class DynamicArray {
private:
    T* data;
    int count; // elements currently stored
    int cap;   // elements the block can hold
    void grow(); // allocate double, copy, delete old
public:
    DynamicArray(int initialCap = 2);
    ~DynamicArray();
    void push_back(const T& value);
    T& at(int index); // must bounds-check
    void removeAt(int index); // shift the tail left
    int size() const;
    int capacity() const;
};
```

When `push_back()` is called and `count` equals `cap`, `grow()` must allocate a new block of double the capacity, copy the existing elements across, delete the old block, and only then store the new element. `at()` must reject an out-of-range index rather than reading past the end of the block.

(b) Observing the Growth

Push the integers 1 to 20 into a `DynamicArray<int>`, printing the capacity every time it changes. Record the sequence of capacities you observe. Explain why the capacity is doubled rather than increased by one each time.

(c) Counting the Cost

- Add a counter that records the total number of element copies performed across all 20 `push_back()` calls — that is, copies done inside `grow()`, not the single store of the new element.
- Divide that total by 20. What is the average number of copies per `push_back()`?
- A single `push_back()` that triggers a `grow` is $O(n)$. Yet the average you just measured stays small and does not grow with n . Explain, in your own words, how both statements can be true at the same time.
- Now suppose `grow()` increased the capacity by 1 instead of doubling it. Derive the total number of copies for n pushes and state the Big-O. Compare it against the doubling strategy.

(d) Putting It to Use

Replace each row of your parking grid with a `DynamicArray<T>`, so the structure becomes an array of dynamic arrays. Then add one menu option:

6. Add a New Slot to a Level

The option should ask for a level number and the occupancy hours of the new slot, and append it to that level. Display the grid afterwards to confirm the level has grown. In a comment, state what your total occupied hours function now costs, and whether it changed from Task 2.

(e) Short Answer

In two or three sentences: for the parking facility, which is the better fit — the raw jagged array of Task 2, or the array of dynamic arrays of Task 3? Consider both memory usage and the cost of adding a slot. There is no single correct answer; the marks are for the reasoning.

Bonus — Only If You Finish Early [+5 marks]

- A Max / Min Occupied Slot report that names the level and slot number, not just the value.
- An Underused Levels report that lists every level whose average falls below a threshold the user enters.
- State the complexity of each report in terms of n .

Marks Distribution

Task	Component	Marks	Where
0	Complexity worksheet, with derivations	10	In lab
1	swapRows and swapRowsByCopy implementation	10	In lab
1	Complexity comparison and justification	5	In lab
2	Jagged allocation, correct destructor, no leaks	12	In lab
2	Menu, input, display in the required format	13	In lab
2	Total and average calculations	8	In lab
2	Input validation	4	In lab
2	Complexity derivation and counter verification	8	In lab
3	DynamicArray class, correct grow and destructor	14	Take home
3	Growth observation and copy counting	8	Take home
3	Integration with the parking grid	5	Take home
3	Short-answer reasoning	3	Take home
Bonus	Max/Min and Underused Levels reports	+5	Optional
	Total	100 (+5)	

Submission Guidelines

- Submit a single file, strictly following the naming convention lXXXXXXX.cpp, using your own roll number and no spaces.
- Place all templated functions and classes above main() in the same file. Do not split templates across files — see Section 3.4.
- Your Task 0 answers may be handed in on paper or included as a comment block at the top of the file, as directed by your lab instructor.
- All complexity derivations for Tasks 1, 2 and 3 must appear in comment blocks immediately above the relevant function.

main() should present a top-level menu that routes to the individual tasks:

```
=== Data Structures Lab 01 ===
1. Task 1 - Row Swap Demonstration
2. Task 2 - Smart Parking Management
3. Task 3 - Dynamic Array Demonstration
0. Exit
Enter choice:
```

- Comment your code. Every function should have a one-line description of what it does.
- Work must be your own. Shared or copied submissions receive zero for all parties involved, in line with university policy.